

Resumo

Esse projeto prevê o desenvolvimento de um datawarehouse baseado em alguma base de dados de exemplo ou corporativa com os seguintes passos:

Parte I

- *Data Warehouse Corporativo/Organizacional, incluindo os Dados Externos etc.*
- *Definir a ETL adequada para o problema*
- *Fazer um relatório explicando o processo e justificando os novos campos calculados ou transformados, desmoralizações etc.*

Parte II

- *A partir do Data Warehouse criado, propor pelo menos 03 modelos de dados para um possível assunto/Data Mart, que podem ser Star Scheme ou Snow Flake*
- *Gerar dashboards no Metabase com os dados dos modelos, para demonstrar o funcionamento.*

Por opção algumas atitudes foram tomadas:

- optou-se por desenvolver um Docker para criação do banco de dados
- optou-se por usar a base de dados do Northwind devida a forças maiores (sem acesso a banco de dados corporativos, vulgo desempregado)
- optou-se por usar o Power BI ao invés do Metabase, visto que trata-se de uma ferramenta “freemium” quando os relatórios são distribuídos publicamente, tenho mais experiência e a modelagem é mais visível.
- foi selecionado o enriquecimento de coordenadas geográficas para incrementar dados de clientes e fornecedores.

Detalhes de criação, iniciação e etc estão expostas no README.md do projeto disponível em: <https://github.com/miguelantoniotoledo/Projeto-Modelagem-de-Dados>

Dashboard no Power BI Service:

<https://app.powerbi.com/view?r=eyJrljoiYzhIMzkxNzUtOWExMy00ZjQ2LWFIMjMtZDI2YzRiODRmMDA5IiwidCI6IjE1MWZmNzU0LWYyYjQ0NDA0OS05M2E5LTdhMTBmM2IwMGQ5NCJ9>

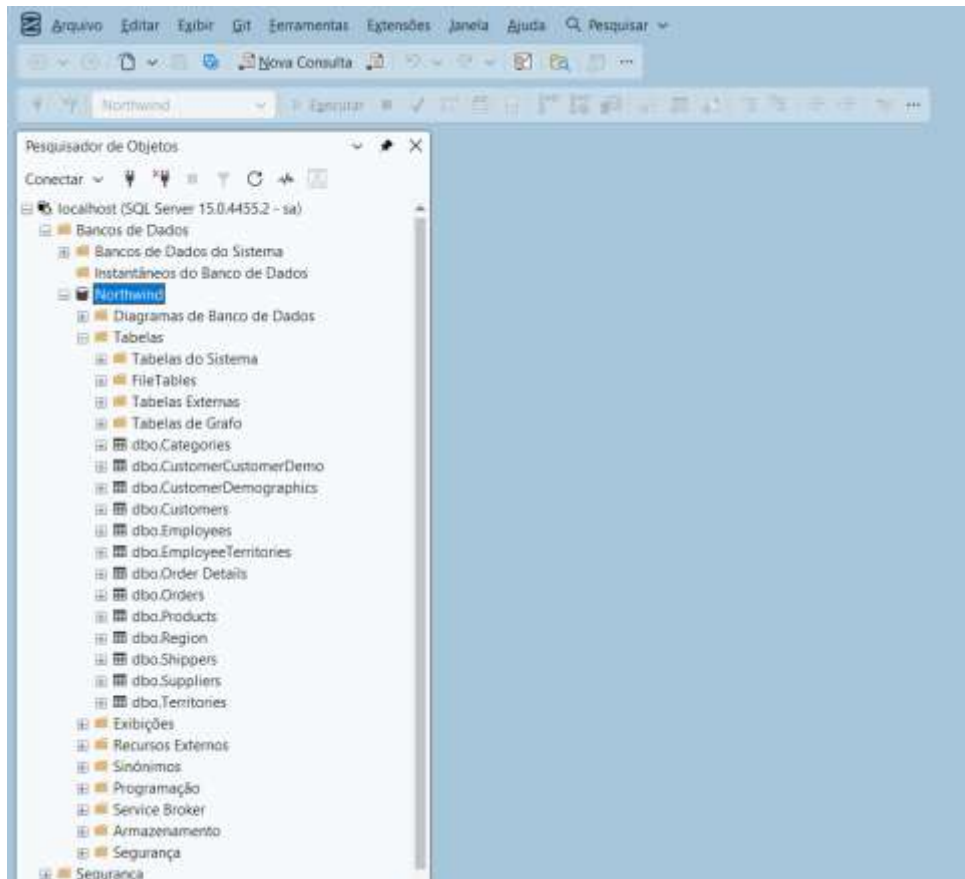
Desenvolvimento do Projeto

Iniciamos um Docker com a imagem do My SQL (verifique mais detalhes no README, tais como configurações e criação do banco):



Configuramos para acesso ao banco a porta 1433 padrão do MYSQL

Criamos o banco de dados da Northwind pelo script disponível no Git do projeto <https://github.com/microsoft/sql-server-samples/blob/master/samples/databases/northwind-pubs/instnwnd.sql>. Usamos para gerenciamento desse banco transacional o SQL Server Management Studio:



Criamos também o dw_northwind para armazenamento do nosso datawarehouse:



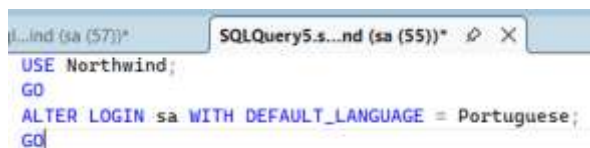
No Visual Studio instalamos o `dbt-sqlserver` para rodar os modelos dentro desse novo datawarehouse. Dessa forma também criamos o projeto dbt com `dbt init dw_northwind`, configurando o `profiles.yml` e o `dbt_project.yml`:



```
! dbt_project.yml X
dw_northwind > ! dbt_project.yml
You, last week | 1 author (You)
1 name: "dw_northwind"
2 version: "1.0.0"
3 profile: "dw_northwind"
4
5 model-paths: ["models"]
6
7 You, last week + commit in

! dbt_project.yml ! profiles.yml X
C: > Users > Acer > .dbt > ! profiles.yml
1 dw_northwind:
2   outputs:
3     dev:
4       database: dw_northwind
5       driver: 'ODBC Driver 18 for SQL Server'
6       host: localhost
7       password: SuaSenhaForte123!
8       schema: dbo
9       port: 1433
10      threads: 4
11      type: sqlserver
12      user: sa
13      encrypt: False
14      trust_cert: True
15      target: dev
16
```

Vamos criar inicialmente as dimensões do modelo. A inicial é a `dim_calendario`, a tabela de datas do modelo. Inicialmente tentou-se usar o package `dbt_utils` com a função `date_spine` para geração das datas, mas sem sucesso. O problema foi ocasionado pelo “engessamento” do banco de dados do SQL Server, que possui pouca flexibilidade na sintaxe. Por fim usamos as funções naturais do banco. Executamos também no banco transacional uma configuração para que os nomes de meses e dias da semana ficassem em português:



```
SQLQuery5.s...nd (sa (55))*
USE Northwind;
GO
ALTER LOGIN sa WITH DEFAULT_LANGUAGE = Portuguese;
GO
```

As datas foram geradas a partir da menor e maior data da tabela `Orders` do transacional:

```
profiles.yml  dim_calendario.sql M X
dw_northwind > models > dim_calendario.sql
You, 3 hours ago | 1 author (You)
1  {{ config(
2    enabled= true,
3    materialized='table'
4  ) }}
5
6  WITH range_datas AS (
7    SELECT
8      MIN(orderdate) AS min_data,
9      MAX(orderdate) AS max_data
10   FROM (
11     source('northwind', 'Orders')
12   )
13 FROM
14   range_datas rg
15 CROSS APPLY (
16   SELECT TOP (DATEDIFF(DAY, rg.min_data, rg.max_data) + 1)
17     ROW_NUMBER() OVER (ORDER BY (SELECT NULL)) - 1 AS number
18   FROM
19     sys.all_columns ac1
20   CROSS JOIN
21     sys.all_columns ac2
22   ) AS n
23 )
24
25 SELECT
26   FORMAT(date_day, 'yyyyMMdd') AS pk_data,
27   CAST(date_day AS DATE) AS data_completa,
28   YEAR(date_day) AS ano,
29   MONTH(date_day) AS mes,
30   DAY(date_day) AS dia,
31   DATENAME(WEEKDAY, date_day) AS dia_semana,
32   DATENAME(MONTH, date_day) AS nome_mes_completo,
33   SUBSTRING(DATENAME(MONTH, date_day), 0, 4) AS nome_mes_abreviado
34 FROM
35   date_spine
```

Depois de construída a `dim_calendario`, vamos normalizar as demais dimensões. A ideia é implantar dimensões na segunda forma normal. Aqui vai uma observação pessoal: muitas vezes precisamos usar algo muito próximo da terceira forma normal devido à dinâmica de filtros (principalmente no Power BI). Ao total construímos as dimensões abaixo, os códigos completos podem ser observados em https://github.com/miguelantoniotoledo/Projeto-Modelagem-de-Dados/tree/main/dw_northwind/models:

```
models
> staging
dim_calendario.sql M
dim_clientes.sql U
dim_fornecedores.sql U
dim_funcionarios.sql U
dim_produtos.sql U
```

Aqui existe um detalhe: dimensionei algumas colunas (como o cálculo da idade do funcionário ou seu tempo de caso) com base na data de 1998, para que todo mundo não seja um idoso na análise.

Construímos também a `fato_vendas` e a `fato_vendas_detalhe`. A primeira contem todos os detalhes mais amplos do da venda, inclusive os valores finais da venda:

```
SELECT
vend.pk_venda,
vend.sk_cliente,
vend.sk_funcionario,
vend.dt_venda,
COALESCE(vend.nm_pais_venda, 'Sem Informacao') as nm_pais_venda,
SUM(det.flt_quantidade) as flt_qnt_produtos,
SUM(det.flt_valor_sem_desconto) as flt_vlr_sem_desconto,
SUM(det.flt_valor_final) as flt_vlr_sem_frete,
SUM(det.flt_valor_final) + MIN(vend.flt_frete) as flt_vlr_com_frete
FROM vendas vend
LEFT JOIN detalhes det
ON vend.pk_venda = det.pk_venda
GROUP BY vend.pk_venda, vend.sk_cliente, vend.sk_funcionario, vend.dt_venda,
```

Na `fato_vendas_detalhe` fazemos a correspondência entre os dados da venda e os detalhes, focando em trazer os produtos, fornecedores e clientes:

```
SELECT
CONCAT(det.OrderId, '-', det.ProductID) as pk_detalhe,
det.OrderID as sk_venda,
det.ProductID as sk_produto,
vend.EmployeeID as sk_funcionario,
vend.CustomerID as sk_cliente,
det.UnitPrice as flt_valor_unit,
det.Quantity as flt_quantidade,
(det.Quantity * det.UnitPrice) as flt_valor_sem_desconto,
(det.Quantity * det.UnitPrice) - (det.Discount * det.UnitPrice) as flt_valor_final
FROM
({
  source('northwind', 'Order Details')
}) as det
LEFT JOIN ({
  source('northwind', 'Orders')
}) as vend
ON det.OrderID = vend.OrderID
```

Garantimos para todos os modelos que as pk (chaves primárias) que ela sejam únicas e não nulas, como por exemplo na `dim_cliente` com `pk_cliente` não nula e não vazia:

```
- name: dim_clientes
description: "tabela de cadastro dos clientes baseada na Customers"
columns:
  - name: pk_cliente
    description: "Chave primaria dessa tabela, sendo no formato string"
    data_tests: You, 19 minutes ago • Uncommitted changes
    - unique
    - not_null
```

Na `dim_produtos` ainda fazemos a garantia de relacionamento com as sk (chaves estrangeiras), assim como na `fato_vendas_detalhe`:

```

- name: dim_produtos
  description: "tabela de cadastro dos produtos, oriundo de Products, Categories e Suppliers"
  columns:
    - name: pk_produto
      description: "Chave primaria dessa tabela, sendo no formato inteiro"
      data_tests:
        - unique
        - not_null
    - name: sk_fornecedor
      description: "Chave estrangeira da dim_fornecedores"
      data_tests:
        - not_null
      relationships:
        to: ref('dim_fornecedores')
        field: pk_fornecedor

- name: fato_vendas_detalhe
  description: "tabela de detalhes das vendas vindo de Orders, Order Details"
  columns:
    - name: pk_detalhe
      description: "Chave composta do modelo, dado por Orders e Products"
      data_tests:
        - unique
        - not_null
    - name: sk_venda
      description: "Chave estrangeira da fato_vendas"
      data_tests:
        - not_null
      relationships:
        to: ref('fato_vendas')
        field: pk_venda

    - name: sk_produto
      description: "Chave estrangeira da dim_produtos"
      data_tests:
        - not_null
      relationships:
        to: ref('dim_produtos')
        field: pk_produto
    - name: sk_funcionario
      description: "Chave estrangeira da dim_funcionarios"
      data_tests:
        - not_null
      relationships:
        to: ref('dim_funcionarios')
        field: pk_funcionario

    - name: sk_cliente
      description: "Chave estrangeira da dim_clientes"
      data_tests:
        - not_null
      relationships:
        to: ref('dim_clientes')
        field: pk_cliente

```

Todos esses detalhes estão configurados em `schema.yml`. Executamos `dbt run` e depois `dbt test` e verificamos que todas as configurações e testes funcionam:

```

[OK in 0.33s]
19:08:56
19:08:56 Finished running 7 table models in 0 hours 0 minutes and 1.12 seconds (1.12s).
19:08:56
19:08:56 Completed successfully
19:08:56
19:08:56 Done. PASS=7 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=7
PS C:\Users\Acer\Documents\Projeto Modelagem de Dados\dw_northwind>

```

```

19:09:17
19:09:17 Finished running 24 data tests in 0 hours 0 minutes and 1.01 seconds (1.01s).
19:09:17
19:09:17 Completed successfully
19:09:17
19:09:17 Done. PASS=24 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=24
PS C:\Users\Acer\Documents\Projeto Modelagem de Dados\dw_northwind>

```

Enriquecimento dos Dados

Para enriquecimento dos dados iremos incluir dados de coordenadas geográficas (latitude e longitude) das tabelas `dim_clientes` e `dim_fornecedores`. Para tal construímos um projeto em python, incluindo as funções que tratam a consulta ao banco (`api_extractor`) e a execução do projeto (`main.py`). Utilizamos a biblioteca `geopy` para consulta:

```

=====
SCRIPT FINALIZADO
=====
PS C:\Users\Acer\Documents\Projeto Modelagem de Dados> python main.py

=====
INÍCIO DO SCRIPT DE GEOCODIFICAÇÃO DE DADOS
=====

--- 🟡 PASSO 1: EXTRAÇÃO DO BANCO DE DADOS ---
Digite o local do seu banco (localhost eh permitido): localhost
Digite o nome do seu database: dw_northwind
Digite o nome do seu usuário do banco de dados: sa
Digite a senha do banco de dados: SuaSenhaForte123!

```

```

## 📄 Dados extraídos
pk_cliente  nm_cidade nm_pais
0          ALFKI      Berlin  Germany
1          ANATR  México D.F.  Mexico
2          ANTON  México D.F.  Mexico
3          AROUT    London    UK
4          BERGS    Luleå    Sweden
..          ...      ...      ...
86         WARTH     Oulu    Finland
87         WELLI    Resende  Brazil
88         WHITC    Seattle  USA
89         WILMK    Helsinki Finland

```

```

--- 🟢 Pipeline Concluido ---

--- 📄 RESULTADOS FINAIS COM COORDENADAS ---

## CLIENTES GEOCODIFICADOS (Amostra)
pk_cliente  nm_cidade nm_pais  latitude  longitude
0          ALFKI      Berlin  Germany  52.517389  13.395131
1          ANATR  México D.F.  Mexico   19.43263  -99.133178
2          ANTON  México D.F.  Mexico   19.43263  -99.133178
3          AROUT    London    UK        51.507446  -0.127765
4          BERGS    Luleå    Sweden    65.583119  22.145954

```

```

✓Atualizados 91 registros em 'dim_clientes'.
✓Atualizados 29 registros em 'dim_fornecedores'.

Conexão com o banco de dados encerrada.

=====
SCRIPT FINALIZADO
=====

```

Resultado no banco de dados:

SQLQuery1.s...nd (sa (58)) 47a064a4e232...fornecedores 47a064a4e232....dim_clientes

1
2
3
4
5
6
7
8
9
10

SELECT TOP (1000) [pk_cliente]
 ,[nm_cliente]
 ,[ds_cargo]
 ,[nm_regiao]
 ,[nm_cidade]
 ,[nm_pais]
 ,[latitude]
 ,[longitude]
FROM [dw_northwind].[dbo].[dim_clientes]

100 % Não foi encontrado nenhum problema

Resultados

Mensagens

	pk_cliente	nm_cliente	ds_cargo	nm_regiao	nm_cidade	nm_pais	latitude	longitude
76	SUPRD	Pascale Cartrain	Accounting Manager	NULL	Charleroi	Belgium	50.4116	4.44453
77	THEBI	Liz Nixon	Marketing Manager	OR	Portland	USA	45.5202	-122.674
78	THECR	Liu Wong	Marketing Assistant	MT	Butte	USA	39.6519	-121.586
79	TOMSP	Karin Josephs	Marketing Manager	NULL	Münster	Germany	51.9625	7.62519
80	TORTU	Miguel Angel Pa...	Owner	NULL	México D.F.	Mexico	19.4326	-99.1332
81	TRADH	Anabela Doming...	Sales Representative	SP	Sao Paulo	Brazil	-23.5507	-46.6334
82	TRAIH	Helvetius Nagy	Sales Associate	WA	Kirkland	USA	47.6765	-122.207
83	VAFFE	Palle Ibsen	Sales Manager	NULL	Århus	Denmark	56.1496	10.2134
84	VICTE	Mary Saveley	Sales Agent	NULL	Lyon	France	45.7578	4.83201
85	VINET	Paul Henriot	Accounting Manager	NULL	Reims	France	49.2578	4.03193
86	WANDK	Rita Müller	Sales Representative	NULL	Stuttgart	Germany	48.7784	9.18001
87	WARTH	Pirkko Koskitalo	Accounting Manager	NULL	Oulu	Finland	65.0118	25.4702
88	WELLI	Paula Parente	Sales Manager	SP	Resende	Brazil	-22.4419	-44.4205
89	WHITC	Karl Jablonski	Owner	WA	Seattle	USA	47.6038	-122.33
90	WILMK	Matti Karttunen	Owner/Marketing A...	NULL	Helsinki	Finland	60.1666	24.9435
91	WOLZA	Zbyszek Piestrze...	Owner	NULL	Warszawa	Poland	52.232	21.0067

SQLQuery2.s...nd (sa (56))

SQLQuery1.sql...ind (sa (58))

47a064a4e232...fornecedores

1

2

3

4

5

6

7

8

9

SELECT TOP (1000) [pk_fornecedor]

, [nm_fornecedor]

, [nm_pais]

, [nm_cidade]

, [nm_regiao]

, [latitude]

, [longitude]

FROM [dw_northwind].[dbo].[dim_fornecedores]

100 %

Não foi encontrado nenhum problema

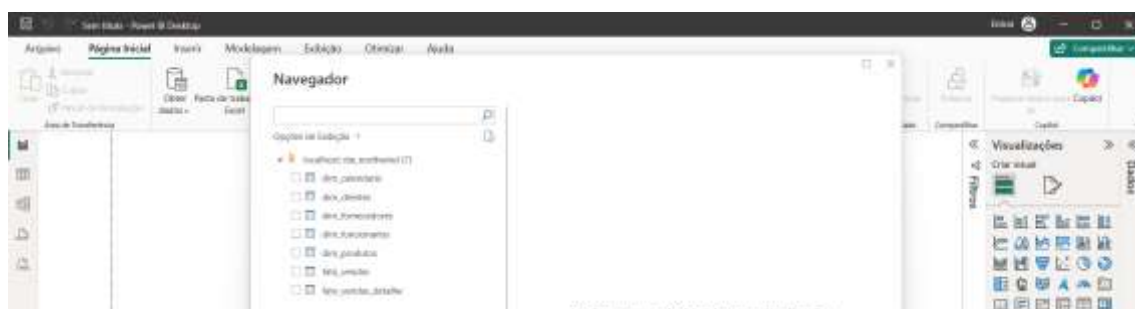
Resultados

Mensagens

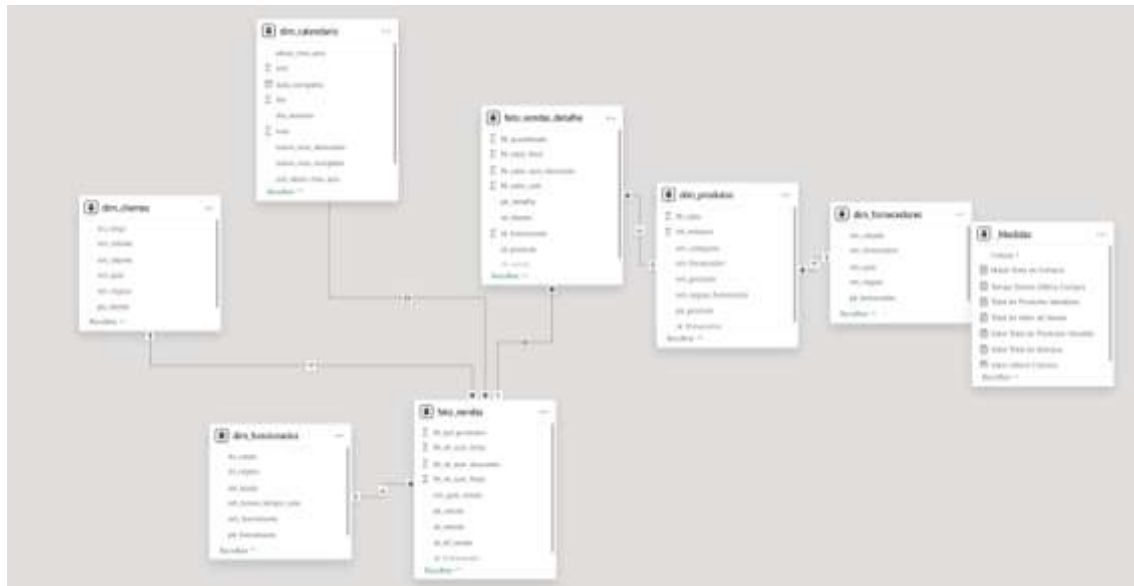
	pk_fornecedor	nm_fornecedor	nm_pais	nm_cidade	nm_regiao	latitude	longitude
1	1	Exotic Liquids	UK	London	Sem Regiao	51.5074	-0.127765
2	2	New Orleans Cajun Delights	USA	New Orleans	LA	29.9559	-90.073
3	3	Grandma Kelly's Homestead	USA	Ann Arbor	MI	42.2814	-83.7485
4	4	Tokyo Traders	Japan	Tokyo	Sem Regiao	35.6769	139.764
5	5	Cooperativa de Quesos 'Las Cabras'	Spain	Oviedo	Asturias	43.3619	-5.84836
6	6	Mayumi's	Japan	Osaka	Sem Regiao	34.6199	135.49
7	7	Pavlova, Ltd.	Australia	Melbourne	Victoria	-37.8142	144.963
8	8	Specialty Biscuits, Ltd.	UK	Manchester	Sem Regiao	53.4795	-2.24511
9	9	PB Knäckebröd AB	Sweden	Göteborg	Sem Regiao	57.7072	11.967
10	10	Refrescos Americanas LTDA	Brazil	Sao Paulo	Sem Regiao	-23.5507	-46.6334
11	11	Heli Süßwaren GmbH & Co. KG	Germany	Berlin	Sem Regiao	52.5174	13.3951
12	12	Plutzer Lebensmittelgroßmärkte AG	Germany	Frankfurt	Sem Regiao	50.1106	8.68209
13	13	Nord-Ost-Fisch Handelsgesellschaft mbH	Germany	Cuxhaven	Sem Regiao	53.8688	8.69829
14	14	Formaggi Fortini s.r.l.	Italy	Ravenna	Sem Regiao	44.3641	12.059
15	15	Norske Meierier	Norway	Sandvika	Sem Regiao	59.8921	10.525
16	16	Bigfoot Breweries	USA	Bend	OR	44.0582	-121.315
17	17	Svensk Sjöföda AB	Sweden	Stockholm	Sem Regiao	59.3251	18.0711

Modelagem e Dataviz

Após isso vamos para o Power BI, desenvolver a modelagem. Configuramos a conexão num plugin padrão para SQL Server:



Ao final da modelagem temos o seguinte diagrama, incluindo as cardinalidades, direção de relacionamento e estrutura geral de relacionamentos. A ideia de trazer a **dim_fornecedores** é baseada na análise das características dos fornecedores e da multiplicidade de fornecedores que um produto pode ter:



Algumas medidas foram criadas para ajudar a montar a análise:

```

1 Maior Data de Compra = MAX(dim_calendario[data_completa])

1 Tempo Desde Ultima Compra =
2 var dataUltima = DATE(1998,5,6)
3 RETURN
4 DATEDIFF([Maior Data de Compra],dataUltima,DAY)

1 Total de Produtos Vendidos = SUM(fato_vendas_detalhe[flt_quantidade])

1 Total de Valor de Venda = SUM(fato_vendas[flt_vlr_sem_frete])

1 Valor Total de Produtos Vendido = SUM(fato_vendas_detalhe[flt_valor_final])

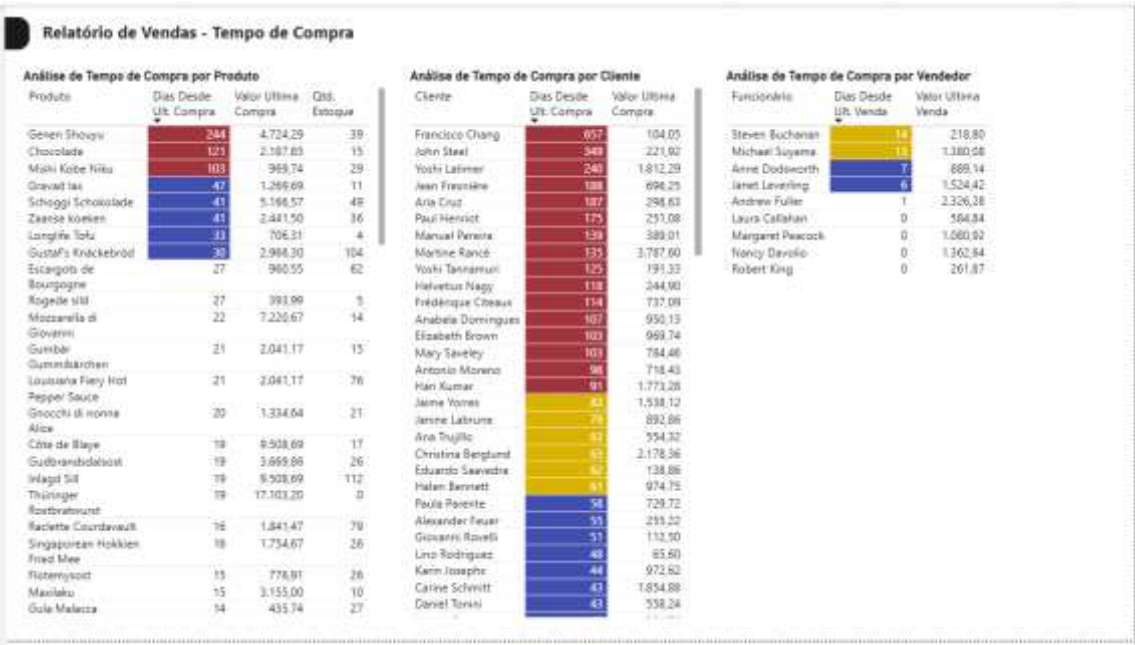
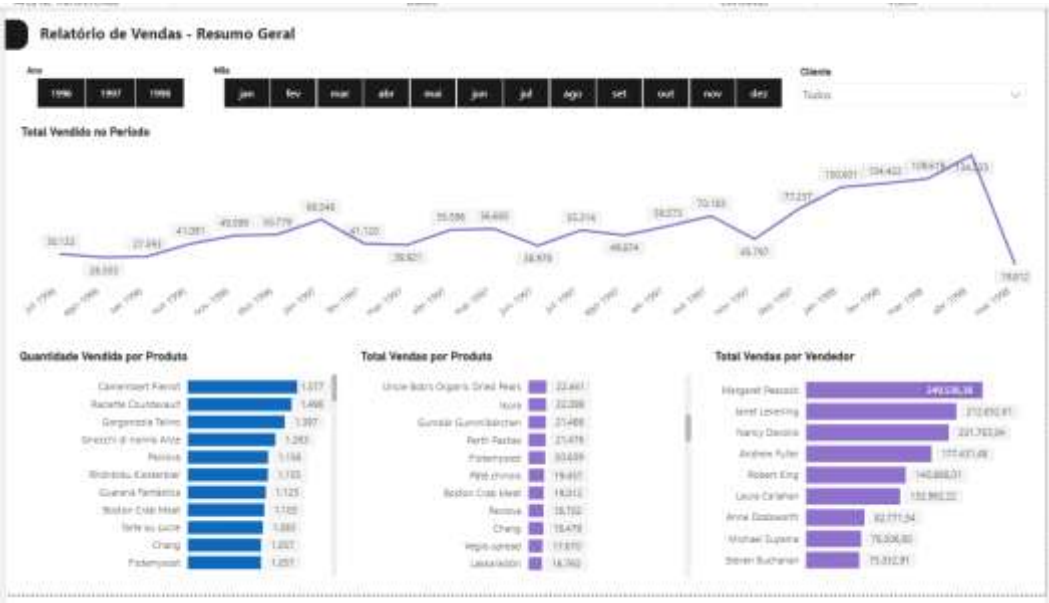
1 Valor Total do Estoque =
2 SUMX(
3     dim_produtos,
4     dim_produtos[int_estoque] * dim_produtos[flt_valor]
5 )

1 Valor Ultima Compra =
2 var maiorData =
3 CALCULATE(
4     MAX(dim_calendario[data_completa]),
5     ALLSELECTED(dim_calendario)
6 )
7 RETURN
8 CALCULATE(
9     SUM(fato_vendas[flt_vlr_com_frete]),
10    dim_calendario[data_completa] = maiorData,
11    REMOVEFILTERS(dim_calendario)
12 )

```

Detalhe na **Tempo Desde Ultima Compra**: setei a data para 06/05/1998 para que o modelo de dados não ficasse muito distante da realidade.

Construí cada umas das visualizações, segmentado ali pelas dimensões. Verifique o título do visual para entender qual dimensão utiliza (se produto, usa dim_produtos e etc):



A ideia não é colocar grandes recursos, “graficar” demais, legendar, explicar: o intuito é mostrar as fatos e dimensões funcionando.

Publicado no Power BI Service. Link de acesso:
<https://app.powerbi.com/view?r=eyJrIjoiYzhIMzIxNzU0OWExMy00ZjQ2LWFIMjMtZDI2YzRiODRmMDA5IiwidCI6Ijk1MWZmNzU0LWYyYjQtNDA0OS05M2E5LTdhMTBmM2IwMGQ5NCJ9>